İLERİ GÖMÜLÜ SİSTEMLER · VİZE PROJESİ

ESP32-S3 ile Raspberry Pi Haberleşmesi ve Eğim Alarmı

Seçilen ekleme: V1 – eğim alarmı, `$ESIK` komutu ve ESP32'de yerel uyarı

Depo: `github.com/gamzesirin/vize-2510140022`

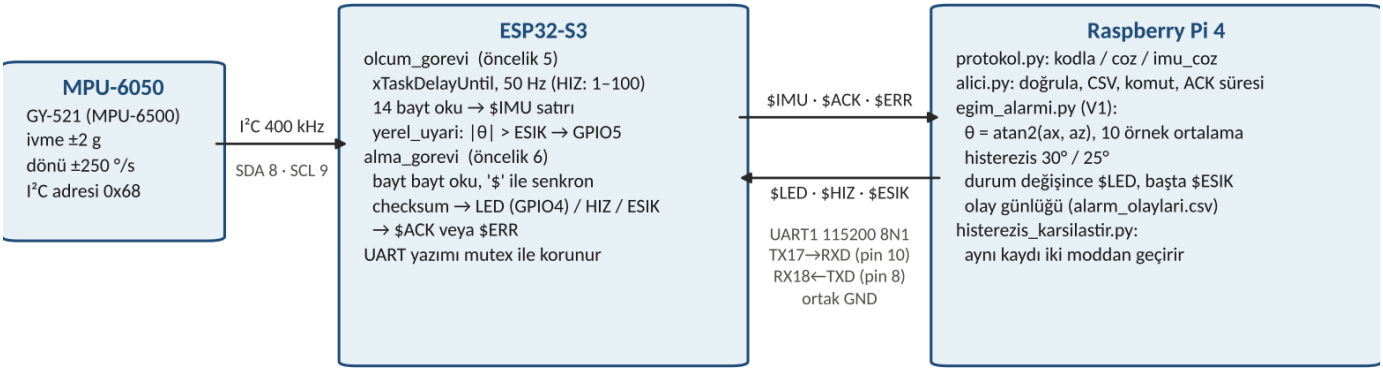
Gamze Şirin · 2510140022
Dr. Murat Sürücü
03.11.2026

Özet. ESP32-S3, MPU-6050 uyumlu sensörü 50 Hz'de okuyup her ölçümü checksum'lı bir metin satırı olarak UART üzerinden Raspberry Pi 4'e gönderir. Pi satırları doğrular, CSV'ye kaydeder ve komut gönderir. Eklemede Pi, eğim açısını hesaplayıp histerezisli bir alarmla ESP32'deki LED'i sürer. Yeni `$ESIK` komutuyla eşik ESP32'ye de bildirilir ve ESP32 ikinci bir LED ile kendi uyarısını verir. 60 saniyelik kayıtta **3001 satırın hiçbiri hatalı değildir** (50,01 Hz), komut–ACK süresi 20 tekrarda **3,4–3,5 ms**'dir. Açı hatası 0°, 45° ve 90°'de **en fazla 0,33°** olmuştur. Açı eşiğin yakınında beklerken histerezis LED değişimini **11'den 1'e** indirmiştir.

1. Amaç

Projenin amacı, zamanlaması kesin olması gereken ölçüm işini mikrodenetleyiciye, karar verme ve kayıt işini Linux çalışan Pi'ye vererek iki kartı güvenilir bir protokolle konuşturmaktır. Temel sistemde ESP32 ölçer ve Pi kaydeder. Ekleme bu veriden bir karar üretir: Pi her örnekten eğim açısını `θ = atan2(ax, az)` ile hesaplar, son 10 örneğin ortalamasını alır, ortalamanın mutlak değeri **30°**'yi geçince alarmı başlatır ve **25°**'nin altına inince bitirir. `$LED` komutu yalnızca alarm durumu değiştiğinde gönderilir ve her olay zaman damgasıyla günlüğe yazılır. Pi başlarken üst eşiği `$ESIK` ile ESP32'ye bildirir. ESP32 de açıyı kendisi hesaplar ve eşiği aşınca GPIO5'teki ikinci LED'i yakar. Böylece Pi bağlantısı koptuğunda da yerel uyarı çalışmaya devam eder.

2. Mimari



Şekil 1. Sistemin blok şeması: görevler, dosyalar ve hatlar.

ESP32'de iki FreeRTOS görevi vardır. `olcum_gorevi` (öncelik 5) `xTaskDelayUntil` ile her 20 ms'de uyanır, sensörün 0x3B'den başlayan 14 baytını okur ve `$IMU` satırını gönderir. Bekleme süresi bir önceki uyanma anından hesaplandığı için okuma ve gönderme süresi periyoda eklenmez. `alma_gorevi` (öncelik 6) UART'tan gelen baytları satır satır toplar, satırı doğrular ve komutu uygular. Daha yüksek öncelikli olduğu için gelen bir komut, bir sonraki ölçümü beklemeden işlenir. İki görev aynı UART'a yazdığından yazma işlemi bir mutex ile korunur; böylece bir ACK satırı bir IMU satırının ortasına girmez. **Pi**'de `protokol.py` satırı kodlar ve çözer, `alici.py` temel alıcıdır. `egim_alarmi.py` ekleme programıdır, `histerezis_karsilastir.py` ise kaydedilmiş bir eğim verisini iki alarm modundan geçiren ölçüm aracıdır. Donanım olmadan deneme için `sim_esp32.py` sanal bir seri port açar.

3. Bağlantı

ESP32-S3	Bağlandığı yer	Not
GPIO17 (UART1 TX)	Pi pin 10 (GPIO15, RXD)	çapraz
GPIO18 (UART1 RX)	Pi pin 8 (GPIO14, TXD)	çapraz
GND	Pi pin 6 (GND)	ortak toprak, breadboard "–" rayı
3V3 / GND	sensör VCC / GND	sensör 3,3 V'tan beslenir
GPIO8 / GPIO9	sensör SDA / SCL	I²C 400 kHz
GPIO4	330 Ω → LED → GND	Pi'nin alarmı ($LED)
GPIO5	330 Ω → LED → GND	ESP32'nin yerel uyarısı (V1)

Tablo 1. Bağlantılar. Kartlar ayrı USB kablolarından beslenir ve Pi'nin 5 V pinleri (2 ve 4) kullanılmaz. Pi'de seri konsol kapatılmış, donanım UART'ı açılmıştır (`/dev/serial0 → ttyS0`).

4. Protokol

Her satır `$GÖVDE*HH\r\n` biçimindedir. `HH`, `$` ile `*` arasındaki karakterlerin ASCII kodlarının XOR'udur ve iki haneli onaltılık sayıyla yazılır; örneğin `$LED,1*50`. Hat 115200 baud, 8N1'dir. Ham ivme 16384 LSB/g (±2 g), ham dönü hızı 131 LSB/(°/s) (±250 °/s) ile birime çevrilir.

Yön	Mesaj	Anlamı
ESP32 → Pi	`$IMU,t,ax,ay,az,gx,gy,gz`	t: ESP32 saati (ms); altı ham ölçüm
Pi → ESP32	`$LED,0|1`	GPIO4 LED'ini söndür / yak
Pi → ESP32	`$HIZ,1..100`	ölçüm hızı (Hz)
Pi → ESP32	**`$ESIK,1..89`**	**V1 – yeni komut:** `|θ|` bu değeri aşınca GPIO5 yanar, 5° altına inince söner
ESP32 → Pi	`$ACK,<komut>,<değer>`	komut uygulandı, ör. `$ACK,ESIK,30`
ESP32 → Pi	`$ERR,BICIM`	$ veya * yok, HH iki hane değil, değer sayı değil ya da aralık dışında, satır 95 karakterden uzun
ESP32 → Pi	`$ERR,CHECKSUM`	HH, gövdenin XOR'u ile tutmuyor
ESP32 → Pi	`$ERR,KOMUT`	bilinmeyen komut adı
ESP32 → Pi	`$ERR,I2C`	sensör okunamadı (hata sürerse 50 ölçümde bir bildirilir)

Tablo 2. Mesajlar ve hata kodları. Pi tarafında biçimi ya da checksum'ı bozuk her satır atılır ve "hatalı satır" olarak sayılır.

5. Ölçüm

5.1 60 saniyelik kayıt (`alici.py --sure 60`, sensör düz ve sabit)

60,0 saniyede **3001 geçerli satır** gelmiş, ortalama örnek hızı **50,01 Hz**, hatalı satır sayısı **0** olmuştur. ESP32'nin zaman damgasına göre ardışık 3000 aralığın tamamı tam **20 ms**'dir. Bu, `xTaskDelayUntil` ile kurulan periyodun kaymadığını gösterir. Satırların Pi'ye varış aralıkları da milisaniye çözünürlükte 19–21 ms arasında kalmıştır (standart sapma 0,045 ms). Ayrıca 196 saniyelik bir alarm kaydında zaman damgalarında 100 ms'yi aşan hiçbir boşluk bulunmamış, yani örnek kaybı olmamıştır (9810 örnek). Düz duran sensörde ivme ax, ay ve az için standart sapma 0,003–0,004 g'dir. Jiroskop dururken +2,16, −0,54 ve −1,31 °/s ölçmüştür; bu değerler sıfır noktası kaymasıdır. Toplam ivme 1,000 g yerine 0,955 g okunmuştur. Bazı kayıtlarda tek bir hatalı satır görülmüştür. Bunun nedeni, program portu açtığında akmakta olan satırın ortasından yakalanmasıdır. Bu yarım satır `$` ile başlamadığı için biçim denetiminde (`BICIM`) yakalanıp atılmaktadır.

5.2 Komut–ACK süresi (`alici.py --ack-testi 20`)

0,3 saniye arayla gönderilen 20 `$LED` komutunun 20'sine de yanıt gelmiştir. 19'u **3,4 ms**'de, biri 3,5 ms'de dönmüştür. Bu sürenin 2,26 ms'si iletim süresidir: 11 baytlık komut ile 15 baytlık ACK toplam 26 bayttır ve 115200 baud'da bayt başına 10 bit ile 2,26 ms sürer. Kalan yaklaşık 1,1 ms, iki tarafın satırı işleme süresine ve Linux'un seri porttan okuma gecikmesine aittir. Sürenin bu kadar sabit olmasının bir nedeni vardır: alıcı komutu her zaman bir IMU satırını aldıktan hemen sonra gönderdiği için ESP32'nin verici hattı o anda boştur. `egim_alarmi.py`'nin başında, rastgele bir anda gönderilen `$ESIK` komutlarında ise kayıtlı dört çalıştırmada ACK süresi **3,8–5,7 ms** arasında değişmiştir. `$ESIK` ve ACK'i toplam 30 bayttır (2,60 ms), yani en kısa değer LED'deki gibi yaklaşık 1,1 ms ek süreyle açıklanır. Aradaki 2 ms'ye kadar fark ise ACK'in o anda iletilmekte olan yaklaşık 45 baytlık bir IMU satırının (3,9 ms) kalan kısmını beklemesiyle uyumludur.

5.3 `hiz` komutunun etkisi

`hiz 10` komutunun ACK'i 3,6 ms'de gelmiş, 5 saniyelik pencerelerde ölçülen hız 9,9–10,1 Hz'e düşmüştür. `hiz 50` ile hız yeniden 50,0 Hz olmuştur. Geçiş penceresinde ara değerler (44,7 Hz ve 34,3 Hz) görülmüştür, çünkü o 5 saniyenin bir kısmı eski, bir kısmı yeni hızla geçmiştir.

5.4 Eğim açısının doğruluğu (`egim_alarmi.py --olc 10 --gercek X`)

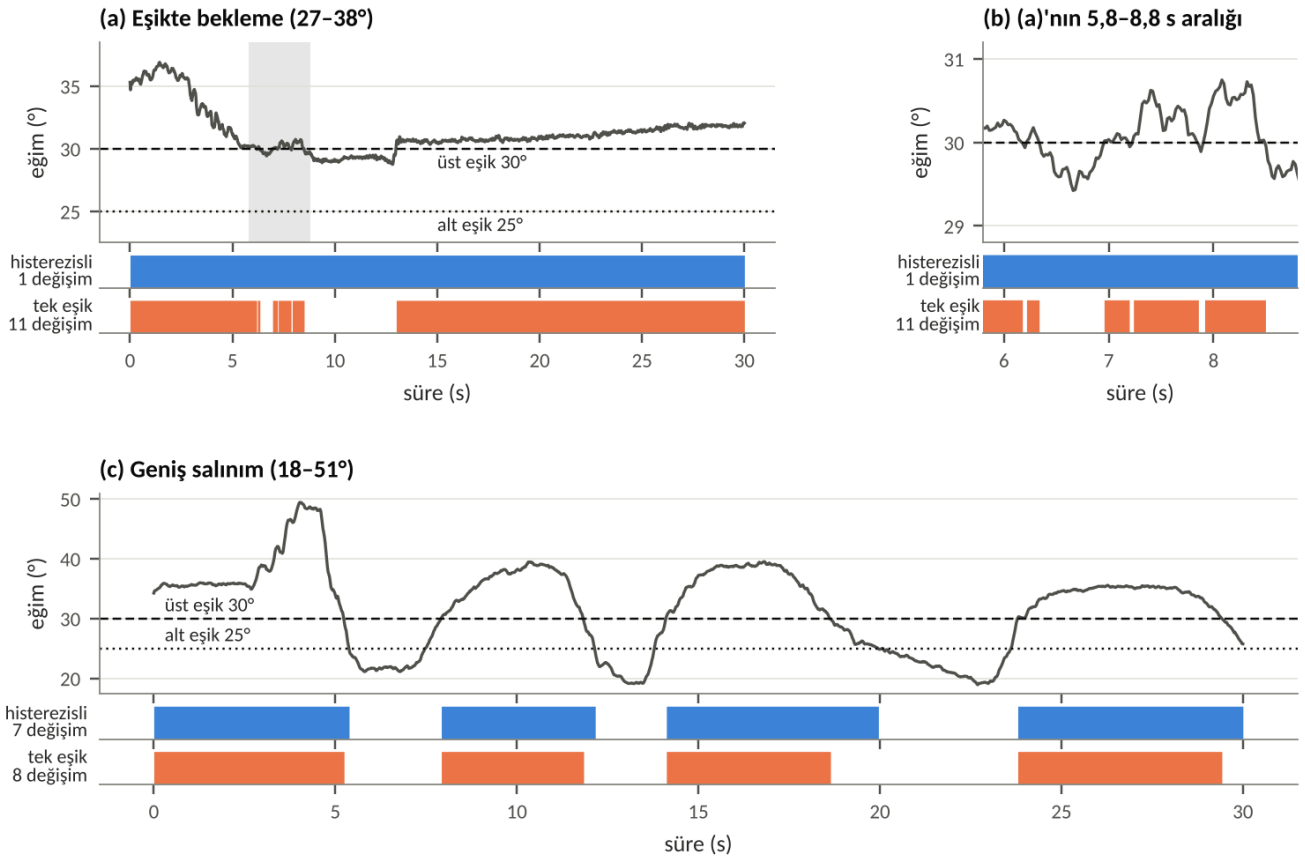
Gerçek açı	Ölçülen ortalama	Hata	Std. sapma	Örnek
0°	+0,10°	+0,10°	0,14°	501
45°	44,67°	−0,33°	0,47°	501
90°	90,00°	0,00°	0,47°	501

Tablo 3. Her açıda 10 saniyelik ölçüm. Referans: düz masa (0°), telefonun su terazisi ile 45°, dik duran kitap (90°).

Üç açıda da hata en fazla 0,33°'dir. Sensör toplam ivmeyi %4,5 düşük okumasına rağmen açı doğru çıkmaktadır. Bunun nedeni, `atan2(ax, az)`'ın iki eksenin **oranını** kullanmasıdır; iki eksene ortak olan bir kazanç hatası bu oranda sadeleşir. 45°'de ilk ölçümde kitap elle tutulmuş (49,25°, standart sapma 2,25°), ikincisinde bir yere yaslanmış ama ölçüm sırasında kaymıştır (48,29°, standart sapma 1,70°; açı 10 s boyunca düzenli olarak azalmıştır). Kitabın alt kenarı dayanıp sabitlendiğinde standart sapma 0,47°'ye inmiş ve hata −0,33° olmuştur. Yani ilk iki ölçümdeki fark sensörden değil, ölçüm düzeneğinin oynamasından gelmektedir; bu ölçümler de depoda tutulmuştur.

5.5 Histerezisin etkisi (`histerezis_karsilastir.py`)

Kartı elle her seferinde aynı şekilde hareket ettirmek mümkün olmadığından iki mod, **aynı kayıt** üzerinde karşılaştırılmıştır. `histerezis_karsilastir.py`, `egim_alarmi.py`'deki `Alarm` sınıfının kendisini kullanarak kaydı iki moddan geçirir. Kayıt sırasında canlı sayılan değişim sayısı, programın yeniden hesapladığı değerle iki kayıtta da aynı çıkmıştır (7 = 7 ve 11 = 11). Açı eşiğin yakınında beklerken (Şekil 2a–b) tek eşikli alarm **11 kez** değişmiştir. Ortalama 29,95° ile 30,10° arasında gidip gelirken 6,2–8,5 s arasında, yani 2,3 saniyede 9 değişim olmuştur. Bu sırada oluşan durumlardan 5'i yarım saniyeden kısa sürmüş, en kısası 40 ms (iki örnek) olmuştur. Histerezisli alarm aynı kayıtta yalnızca **1 kez** değişmiştir. Açı iki eşiği de aşacak kadar geniş salındığında (Şekil 2c) fark küçüktür (7'ye karşı 8). Bu beklenen bir sonuçtur, çünkü o durumda alarmın gerçekten açılıp kapanması gerekir. Histerezis yalnızca eşik çevresindeki titreşimi bastırır.



Şekil 2. Aynı kaydın 10 örneklik ortalaması (gri) ve iki alarm modunun LED durumu (mavi: histerezisli 30°/25°, turuncu: tek eşik 30°). (a) Eşiğin yakınında bekleme, (b) bunun 5,8–8,8 s aralığı, (c) iki eşiği de aşan geniş salınım. Şekiller `rapor/sekil_ciz.py` ile `olcum/` kayıtlarından çizilmiştir.

ESP32'nin yerel uyarısı: Varsayılan 30° eşikle GPIO5 LED'i 30,4°'de yanmış, 24,1°'de sönmüştür. `esik 45` komutundan sonra (ACK 3,8 ms) 30°'de yanmamış, 45,5°'de yanmış ve 39,4°'de sönmüştür. `esik 99` aralık dışında olduğu için `$ERR,BICIM` ile reddedilmiştir. Üç turluk alarm denemesinde iki LED birlikte yanıp sönmüş, altı olayın hepsi `olcum/alarm_olaylari.csv`'ye zaman damgasıyla yazılmıştır.

6. Sorunlar

- **İlk komutun `$ERR,BICIM` alması.** Pi yeniden başlarken ESP32 açıktı ve ilk `led 1` komutu `BICIM` hatası aldı; ikinci komut ise çalıştı. ESP32'nin alım kodu incelendiğinde satırın yalnızca `\n` ile bittiği görüldü. Pi açılırken hattan gelen, satır sonu olmayan bir yarım veri tamponda kalmış ve ilk komut bu verinin arkasına eklenmişti. Birleşen satır `$` ile başlamadığı için reddedilmişti. **Çözüm:** `$` karakteri artık her zaman yeni bir satır başlatıyor; öncesindeki yarım veri atılıyor ve ESP32 log'una uyarı olarak yazılıyor (ayrı "Düzeltme" commit'i). Düzeltmeden sonra aynı durum Pi'den `printf 'cop' > /dev/serial0` ile bilerek oluşturuldu; komut bu kez doğrudan ACK almıştır (3,8 ms).
- **Bölünmüş GND rayı.** Breadboard'un yan rayları 30. sütun hizasında ikiye ayrılıyor. Pi'nin GND kablosu ilk kurulumda rayın diğer yarısına takılmıştı ve ortak toprak oluşmuyordu. Kablolama güç verilmeden önce fotoğraf üzerinden kontrol edilirken fark edildi ve kablo ESP32 ile sensörün GND'si ile aynı yarıya taşındı.
- **Sensörün kimliği.** GY-521 kartından WHO_AM_I olarak MPU-6050'nin 0x68 değeri yerine 0x70 okundu; yani kartta MPU-6500 vardır. Kod iki değeri de kabul edecek şekilde yazıldı. Kullanılan registerların adresleri (0x3B'den başlayan 14 bayt, 0x1A–0x1C, 0x6B) iki yongada aynıdır. Fark şudur: MPU-6500'de ivmeölçerin süzgeci ayrı bir registerdadır (0x1D), bu yüzden 0x1A'ya yazılan 0x03 bu kartta yalnızca jiroskobu süzer.
- **Açı ölçümünde yanlış eksen.** İlk açı denemelerinde kart 45° eğik olduğu halde 2–6,5° ölçüldü. `θ = atan2(ax, az)` yalnızca Y ekseni etrafındaki dönmeyi, yani sensörün X ekseninin ne kadar yukarı kalktığını ölçer; kart buna dik yönde eğilmişti. Bazı denemeler yanlış gerçek açıyla kaydedilmiş, birinde kart ters döndüğü için açı ±180° civarında çıkmıştı. Bir denemede de sensör okunamadı: 10 kez `$ERR,I2C` geldi ve yalnızca 22 örnek kaydedildi. Kart kitaba yerleştirilirken bir kablonun bir an temasını kaybettiği düşünülmektedir; ESP32 yeniden başlatıldıktan sonra hata tekrarlanmadı. Bu 14 deneme `olcum/aci_gecersiz_denemeler.csv`'ye ayrıldı. Ölçümler, eğim yönü canlı okumayla kontrol edilip kart sabitlenerek tekrarlandı.

Beyan: Bu projedeki ESP32 ve Pi kodlarının (`main.c`, `protokol.py`, `alici.py`, `sim_esp32.py`, `egim_alarmi.py`, `histerezis_karsilastir.py`) yazımında, şekillerin çiziminde ve bu raporun taslağının hazırlanmasında yapay zekâ asistanından (Claude) yararlanılmıştır. Devre tarafımdan kurulmuş, kodlar tarafımdan derlenip donanımda çalıştırılmış ve tüm ölçümler tarafımdan yapılmıştır.